



Sustainable Food Ordering Automation Project

Building scalable, observable, and robust data automation for efficient order processing in sustainable food markets.

The Challenge & Solution

The Challenge

In the growing market of sustainable food, managing and processing orders efficiently is a significant challenge. Manual processes are slow, error-prone, and can't scale to meet increasing demand.

Our Solution

We've built a viable data automation system to streamline the food ordering process. Our platform is designed to be scalable, observable, and robust, using modern DevOps and data engineering tools.



Project Roadmap

01

Project Overview & Architecture

03

Deployment & Operations

02

Core Technologies & Trade-offs

04

Future Architecture & Scaling

Architecture: Monolithic MVP

Our application combines distinct logical parts—order service and data pipeline—into a single deployable unit.

Pros

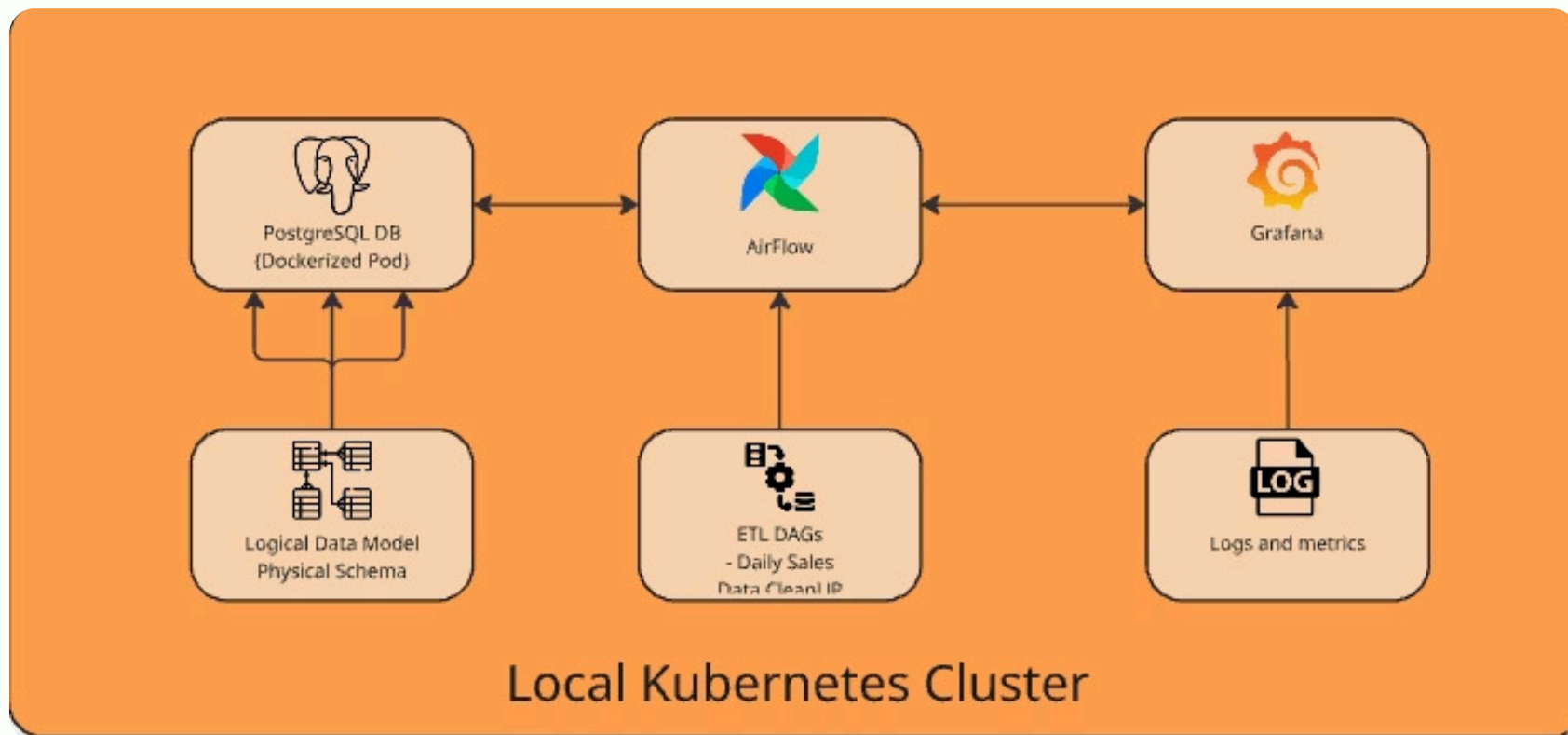
Simplicity & Speed: It allowed us to develop and launch much faster without the initial complexity of a distributed system.

Cons

Scaling Limitations: We have to scale the entire application together, which is inefficient. A bug in one part (like reporting) could potentially bring down the whole system. This architecture is a starting point, not the final destination.



Monolith Architecture





Containerization with Docker

Docker packages our application and its dependencies into lightweight, portable units called containers, using a Dockerfile as its blueprint.

Pros: Consistent Environments

Docker solves the "it works on my machine why not on yours ?" problem, ensuring consistent execution across development, testing, and production environments. Containers also offer greater resource efficiency than traditional virtual machines by sharing the host OS kernel.

Cons: Orchestration Complexity

Managing container networking, persistent storage, and the lifecycle of numerous containers at scale becomes complex without an orchestration tool like Kubernetes. This adds a layer of operational overhead.

Orchestration with Kubernetes

Kubernetes (K8s) is an open-source platform that automates the deployment, scaling, and management of containerized applications, orchestrating where and how they run across a cluster.

Pros: Resilience & Scalability

K8s ensures high availability through self-healing (auto-restarting failed containers) and auto-scaling. It also abstracts infrastructure, providing a consistent deployment target across environments.

Cons: Complexity & Learning Curve

K8s is complex with a steep learning curve. We chose it over simpler alternatives for its powerful ecosystem and long-term scalability, critical for our future growth.





ETL & Workflow Automation with Apache Airflow

Apache Airflow is our chosen platform to programmatically author, schedule, and monitor data workflows. We define our data pipelines as Directed Acyclic Graphs (DAGs) in Python.



Pros: Robust Automation

- **Workflows-as-Code:** Ensures pipelines are versionable, testable, and collaborative.
- **Rich UI:** Excellent for monitoring and managing complex workflows.
- **Extensive Integrations:** A broad provider ecosystem offers flexibility with various data tools.



Cons: Batch Limitation

- **Batch-Oriented:** Airflow is fundamentally a batch scheduler, not designed for low-latency, real-time stream processing.
- **Future Challenge:** This limitation will be a key architectural consideration as we expand into real-time data needs.

Data & Observability

PostgreSQL

Pro: ACID compliance guarantees data integrity for order processing.

Con: Horizontal scaling is challenging; single instance can become a bottleneck.

Grafana

Pro: Versatile visualization connecting to any data source for unified monitoring.

Con: Requires pairing with metrics collection (Prometheus) and log aggregation (Loki/ELK).



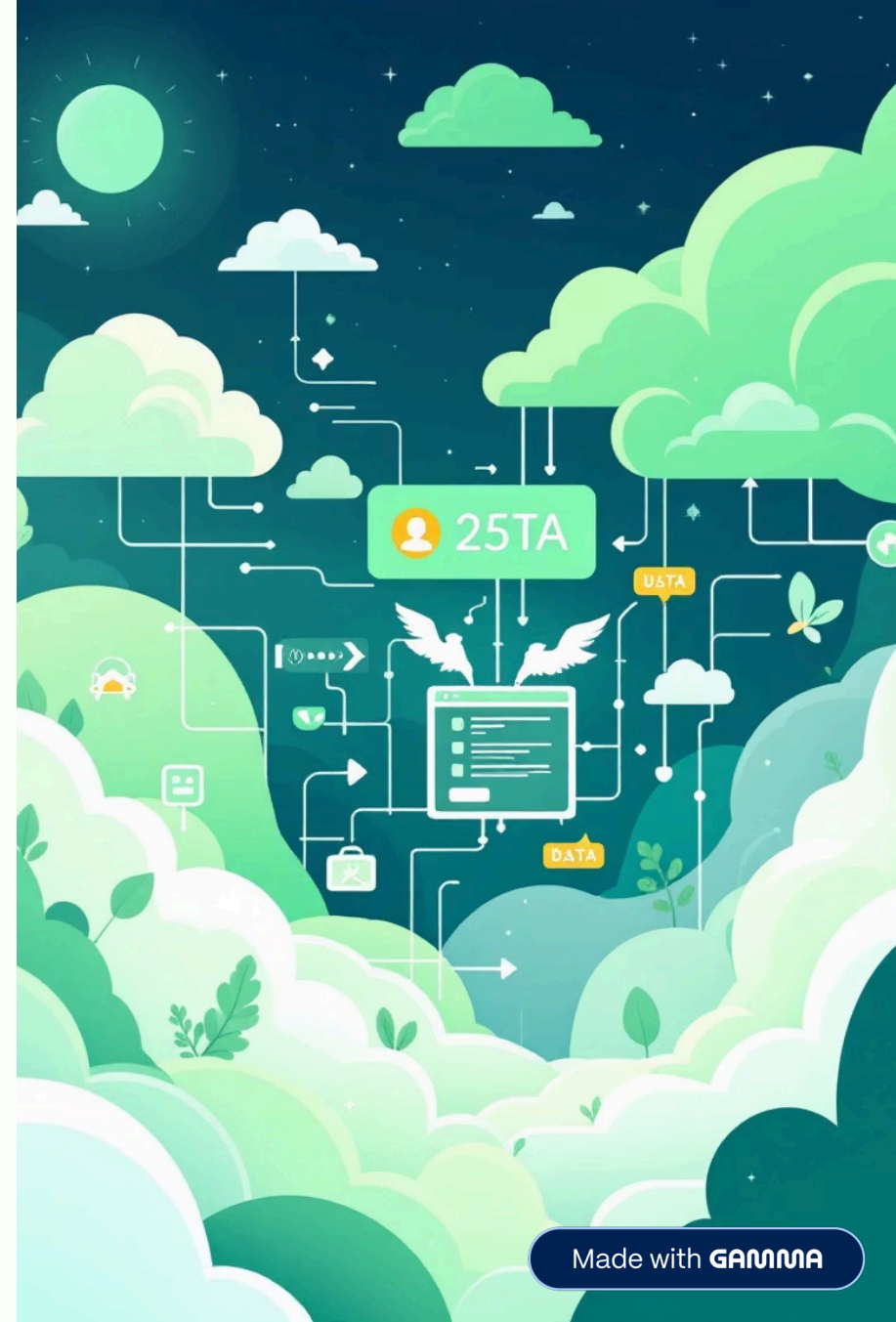
Evolution Roadmap: Steps 1-2

Step 1: Microservices

Decompose monolith into independent services (Orders, ETL, Notifications). Enable independent scaling and improve fault tolerance.

Step 2: Event-Driven

Introduce Apache Kafka. Services publish events; others subscribe. Decouple dependencies, improve resilience, enable real-time streams.





Evolution Roadmap: Steps 3

1

Real-Time Processing

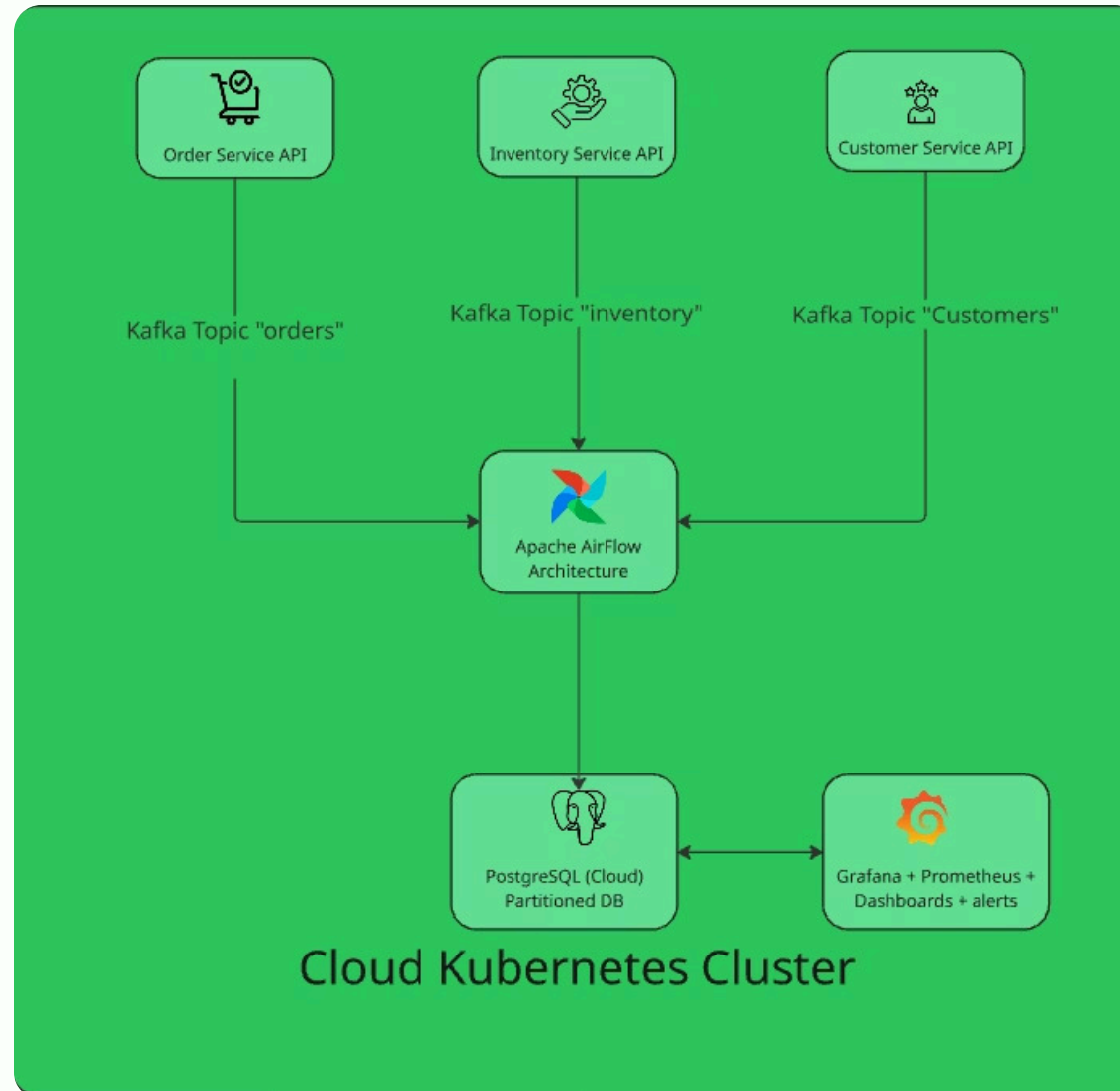
Stream processing with Spark Streaming or Flink. Live analytics dashboards powered by Kafka events.

2

Data Scaling

PostgreSQL read replicas for distributed reads.
Dedicated Data Warehouse (Snowflake/BigQuery) separates analytics from operations.

Cloud Kubernetes Cluster



Conclusion

We've built a functional MVP using industry-standard tools. Our monolithic start enabled rapid development, while Kubernetes provides a powerful foundation for evolution.

- ❏ **Our Path Forward:** Transition to a decoupled, event-driven microservices architecture. Enable real-time processing and advanced MLOps with Kubeflow.



Thank You!

Building the future of sustainable food automation together.